LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2
MODUL 4
“PROSEDUR”

DISUSUN OLEH:

RAFI RAMADHAN

109082500140

S1 IF-13-02

DOSEN:

Wahyu Andi Saputra, S.Pd., M.Eng.

PROGRAM STUDI S1 TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2025/2026

DASAR TEORI

Prosedur dapat dianggap sebagai potongan beberapa instruksi program menjadi suatu instruksi baru yang dibuat untuk mengurangi kerumitan dari kode program yang kompleks pada suatu program yang besar. Prosedur akan menghasilkan suatu akibat atau efek langsung pada program ketika dipanggil pada program utama. Suatu subprogram dikatakan prosedur apabila tidak ada deklarasi tipe nilai yang dikembalikan , dan tidak terdapat kata kunci return dalam badan subprogram tersebut. Kedudukannya sama seperti instruksi dasar yang sudah ada sebelumnya (seperti assignment) atau instruksi dari paket lain. Selain itu, penamaan prosedur disarankan menggunakan kata kerja atau sesuatu yang merepresentasikan suatu proses.

Deklarasi dan Pemanggilan Prosedur Penulisan deklarasi sebuah prosedur selalu berada di luar blok dari program utama (seperti func main() pada bahasa Go), dan dapat ditulis baik sebelum maupun setelah blok program utama tersebut. Suatu prosedur hanya akan mulai dieksekusi apabila ia dipanggil secara langsung maupun tidak langsung (melalui perantara subprogram lain) oleh program utama. Pemanggilan suatu prosedur cukup dilakukan dengan menuliskan nama prosedur beserta parameter atau argumen yang diminta.

Konsep dan Jenis Parameter Sebuah subprogram yang dipanggil dapat saling berkomunikasi dengan pemanggilnya melalui argumen yang diberikan pada parameter subprogram tersebut. Berdasarkan letak penulisannya, parameter dibagi menjadi dua, yaitu parameter formal dan parameter aktual. Parameter formal adalah parameter yang ditulis pada saat deklarasi suatu subprogram, yang berfungsi sebagai petunjuk mengenai argumen apa saja yang diperlukan pada saat pemanggilan subprogram. Sedangkan parameter aktual adalah argumen riil yang digunakan pada bagian parameter saat pemanggilan subprogram. Banyaknya argumen serta tipe data pada parameter aktual wajib mengikuti kerangka dari parameter formal.

Alokasi Memori pada Parameter Berdasarkan cara alokasi memorinya, parameter dikelompokkan menjadi pass by value dan pass by reference. Pada metode pass by value, nilai pada parameter aktual disalin ke variabel lokal di subprogram, yang berarti parameter aktual dan formal disimpan di memori komputer dengan alamat yang berbeda. Hal ini membuat subprogram tidak dapat mengembalikan perubahan informasi ke pemanggil melalui parameter aktualnya. Sebaliknya, pada pass by reference, parameter formal berperan sebagai pointer yang langsung menyimpan alamat memori dari parameter aktual. Dengan demikian, setiap perubahan nilai pada parameter formal akan berdampak langsung pada parameter aktual, dan nilai terakhirnya dapat diketahui oleh pemanggil. Penggunaan metode pass by reference ini sangat disarankan untuk diterapkan pada prosedur karena prosedur pada dasarnya tidak bisa mengembalikan nilai (return) ke pemanggilnya secara langsung.

SOAL LATIHAN

Prosedur

1. **Kombinasi dan Permutasi**

Source Code:

```
package main

import "fmt"

func factorial(n int) int {
    if n == 0 || n == 1 {
        return 1
    }
    return n * factorial(n-1)
}

func permutation(n, r int) int {
    return factorial(n) / factorial(n-r)
}

func combination(n, r int) int {
    return factorial(n) / (factorial(r) * factorial(n-r))
}

func main() {
    var a, b, c, d int
    fmt.Scan(&a, &b, &c, &d)

    fmt.Println(permutation(a, c), combination(a, c))

    fmt.Println(permutation(b, d), combination(b, d))
}
```

Output:

```
PS C:\alpro2> go run soal1.go
5 10 3 10
60 10
3628800 1
PS C:\alpro2> go run soal1.go
8 0 2 0
56 28
1 1
```

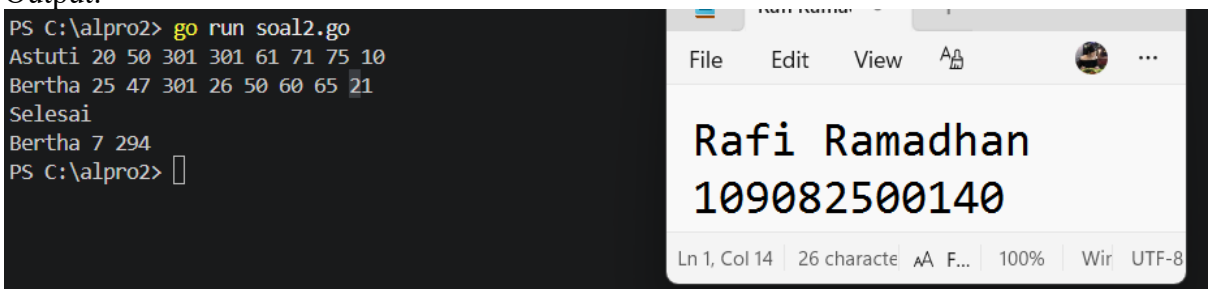
Deskripsi Program:

2. **Gama**

Source Code:

```go
package main

import "fmt"

func hitungSkor(soal *int, skor *int) {
    *soal = 0
    *skor = 0
    var waktu int

    for i := 0; i < 8; i++ {
        fmt.Scan(&waktu)

        if waktu < 301 {
            *soal++
            *skor += waktu
        }
    }
}

func main() {
    var nama, namaPemenang string
    var soalPeserta, skorPeserta int

    var maxSoal = -1
    var minSkor = 999999

    for {
        fmt.Scan(&nama)

        if nama == "Selesai" {
            break
        }

        hitungSkor(&soalPeserta, &skorPeserta)

        if soalPeserta > maxSoal || (soalPeserta == maxSoal &&
skorPeserta < minSkor) {
            maxSoal = soalPeserta
            minSkor = skorPeserta
            namaPemenang = nama
        }
    }

    fmt.Println(namaPemenang, maxSoal, minSkor)
}
```

Output:



Deskripsi Program:

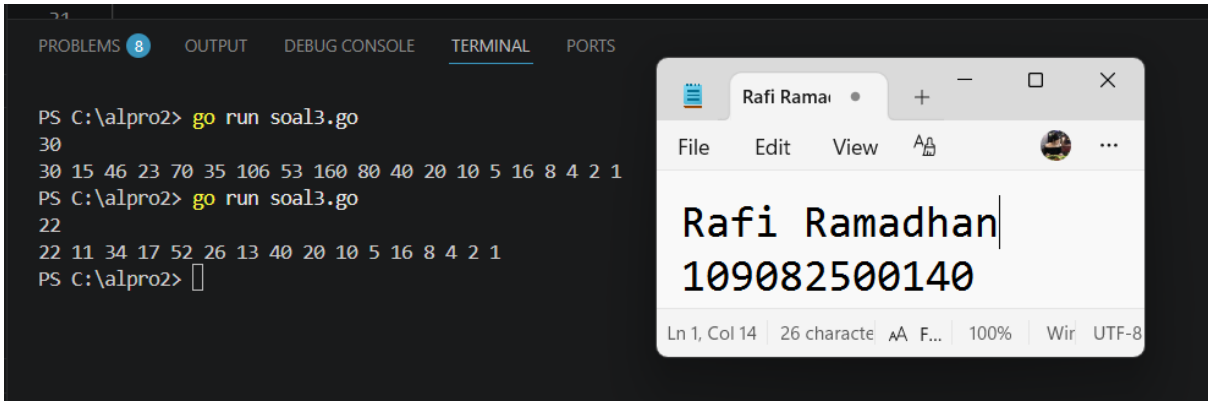
program ini membaca nama satu per satu dan langsung menyerahkan tugas menghitung soal ke dalam prosedur hitungSkor. Setelah prosedur selesai memfilter waktu yang di bawah 301 menit lewat jalur pointer, program utama tinggal membandingkan hasil tersebut dengan rekor juara sementara secara logis hingga data habis.

3. **Skiena dan Revilla**

Source Code:

```
package main

import (
    "fmt"
)

func cetakDeret(n int) {
    for {

        fmt.Printf("%d ", n)


        if n == 1 {
            break
        }


        if n%2 == 0 {
            n = n / 2
        } else {
            n = 3*n + 1
        }
    }
    fmt.Println()
}

func main() {
    var n int

    fmt.Scan(&n)

    if n > 0 && n < 1000000 {
        cetakDeret(n)
    }
}
```

Output:



Deskripsi Program:

Kesimpulan: Program ini mengimplementasikan algoritma deret bilangan (3n+1) menggunakan struktur kontrol perulangan dan percabangan bersyarat dalam bahasa Go. Melalui prosedur cetakDeret, program terus mengevaluasi nilai n dibagi 2 jika genap, dan dikali 3 lalu ditambah 1 jika ganjil. Proses evaluasi dan pencetakan suku bilangan ini terus diulang hingga mencapai base case atau kondisi berhenti, yaitu saat suku terakhir bernilai 1

DAFTAR PUSTAKA

Fakultas Informatika Universitas Telkom. *Modul 4. Prosedur*. Modul Praktikum Algoritma dan Pemrograman 2.